

# Smart Turntable Assistant

Daniel Kiss

2026. április 18.

# Tartalomjegyzék

|          |                                                                    |          |
|----------|--------------------------------------------------------------------|----------|
| <b>1</b> | <b>Bevezetés</b>                                                   | <b>2</b> |
| 1.1      | Témaválasztás indoklása . . . . .                                  | 2        |
| 1.2      | A megoldandó feladat leírása . . . . .                             | 2        |
| <b>2</b> | <b>Felhasználói dokumentáció</b>                                   | <b>4</b> |
| 2.1      | A megoldott probléma megfogalmazása . . . . .                      | 4        |
| 2.2      | A felhasznált módszerek rövid leírása . . . . .                    | 4        |
| 2.3      | A program használatához szükséges információk . . . . .            | 5        |
| 2.3.1    | Telepítés és Indítás . . . . .                                     | 5        |
| 2.3.2    | Alapvető Funkciók Használata . . . . .                             | 5        |
| 2.3.3    | Külső Integrációk . . . . .                                        | 5        |
| <b>3</b> | <b>Fejlesztői dokumentáció</b>                                     | <b>6</b> |
| 3.1      | A probléma részletes specifikációja . . . . .                      | 6        |
| 3.1.1    | Funkcionális követelmények . . . . .                               | 6        |
| 3.1.2    | Nem-funkcionális követelmények . . . . .                           | 7        |
| 3.2      | Felhasznált módszerek és fogalmak . . . . .                        | 7        |
| 3.2.1    | Alapvető fogalmak . . . . .                                        | 7        |
| 3.2.2    | Alkalmazott módszerek . . . . .                                    | 7        |
| 3.3      | Szoftver architektúra és szerkezet . . . . .                       | 7        |
| 3.3.1    | Logikai és fizikai szerkezet . . . . .                             | 7        |
| 3.3.2    | Modellezés és Adatbázis . . . . .                                  | 8        |
| 3.4      | Felhasználói felület tervezése . . . . .                           | 8        |
| 3.5      | Tesztelési terv és eredmények . . . . .                            | 9        |
| 3.5.1    | Tesztelési módszerek . . . . .                                     | 9        |
| 3.5.2    | Tesztelési eredmények . . . . .                                    | 9        |
| 3.6      | Mesterséges intelligencia alkalmazása a fejlesztés során . . . . . | 9        |

# 1. fejezet

## Bevezetés

### 1.1 Témaválasztás indoklása

Az elmúlt években a zeneiparban megfigyelhető a fizikai, analóg hordozók, különösen a bakelitlemezek népszerűségének jelentős visszatérése. Az audiofileket és a hétköznapi hallgatókat egyaránt vonzza a mechanikus élmény és az analóg hangzás egyedi jellemzői. Ugyanakkor a modern digitális zenehallgatási környezet (mint a Spotify vagy az Apple Music) és a hagyományos lemezjátszó használata két teljesen különálló világ maradt.

A témát azért választottam, mert létezik egy jelentős technológiai szakadék az analóg audio jel és a modern metaadat-kezelés között. Egy olyan eszköz létrehozása, amely képes hidat képezni a fizikai lemezjátszó és a digitális kényelmi funkciók között, nemcsak technikai kihívást jelent a valós idejű jelfeldolgozás terén, hanem valódi hozzáadott értéket nyújt a hobbizáló zenehallgatók számára, akik nem szeretnék lemondni az analóg élményről, de igényelnék a digitális kor előnyeit.

### 1.2 A megoldandó feladat leírása

A projekt célja egy olyan „Okos Lemezjátszó Asszisztens” szoftveres rendszer megvalósítása, amely passzívan figyeli egy szabványos lemezjátszó audio kimenetét, és automatikusan nyújt digitális kiegészítéseket a hallgatáshoz.

A rendszernek a következő alapvető feladatokat kell megoldania:

- **Automatikus felismerés:** A bejövő analóg jel alapján meg kell határozni, hogy egy zeneszám kezdődött-e, majd audio fingerprinting segítségével azonosítani kell a pontos előadót és dalcímet.

- **Metaadat-szinkronizáció:** A felismerést követően a rendszernek lekérdeznie kell a számhoz tartozó albumborítót és a szinkronizált dalszöveget, majd azt valós időben megjelenítenie kell a felhasználónak.
- **Hardveres monitorozás:** A szoftvernek matematikai elemzéssel nyomon kell követnie a hangszedő (tű) kopását a lejátszási órák alapján, valamint diagnosztikai adatokat (pl. motorrezgés, felületi zaj) kell szolgáltatnia.
- **Digitális naplózás:** A hallgatási élményt automatikusan szinkronizálnia kell külső platformokkal (pl. Last.fm), így az analóg lejátszás is digitális nyomon követhetővé válik.

## 2. fejezet

# Felhasználói dokumentáció

### 2.1 A megoldott probléma megfogalmazása

A hagyományos bakelitlemez-lejátszó legnagyobb hátránya, hogy a generált jel egy nyers elektromos jel, amely nem tartalmaz metaadatokat. Ez azt jelenti, hogy a felhasználó számára nincs automatikus módja a lejátszott számok naplózására, nincs hozzáférése a szinkronizált dalszövegekhez, és nincs pontos eszköze a tű kopásának vagy a lemezjátszó technikai állapotának ellenőrzésére. A Smart Turntable Assistant ezen „információs vakpontot” törli el egy intelligens szoftverréteg bevezetésével.

### 2.2 A felhasznált módszerek rövid leírása

A rendszer egy kliens-szerver architektúrán alapul, ahol a backend és a frontend különálló egységekben működnek:

- **Audio elemzés és azonosítás:** A Python nyelvű backend a `sounddevice` és `NumPy` könyvtárakat használja a jel feldolgozásához. A számok azonosításához külső Audio Fingerprinting API-kat (pl. ACR-Cloud/Shazam) alkalmaz a rendszer, amelyek egyedi akusztikai „ujjlenyomatokat” hasonlítanak össze egy globális adatbázissal.
- **Valós idejű kommunikáció:** A szerver és a felhasználói felület közötti adatcsere WebSockets (Socket.IO) technológiával történik, ami lehetővé teszi, hogy a diagnosztikai adatok és a dalszövegek késleltetés nélkül érkezzenek a képernyőre.
- **Felhasználói felület:** Egy modern, reaktív weboldal (Vue.js SPA), amely bármilyen eszközön (mobil, asztali) elérhető és dinamikusan változtatja a megjelenítést a lejátszott album borítójához.

- **Adatkezelés:** Az SQLite adatbázis tárolja a felhasználói beállításokat és a tű élettartomát, így a rendszerbe jelentkezve a felhasználó mindig a saját konfigurációját kapja vissza.

## 2.3 A program használatához szükséges információk

### 2.3.1 Telepítés és Indítás

A program Docker konténerben fut, ami egyszerűsíti a telepítést. A futtatáshoz szükség van egy USB audio interfészre, amelyhez csatlakoztatva van a lemezjátszó.

1. A Docker image letöltése és a konténer indítása a mellékelt `docker-compose.yml` fájlal.
2. A böngészőben a `http://localhost:5000` (vagy a szerver IP címe) megnyitása.

### 2.3.2 Alapvető Funkciók Használata

- **Zenehallgatás:** Helyezze a tűt a lemezre. A rendszer automatikusan érzékeli a hangot, azonosítja a számot és megjeleníti az albumborítót valamint a dalszöveget.
- **Hangszedő regisztrálása:** A profil menüben adja meg az új tű típusát és az ajánlott élettartamát. A rendszer ettől kezdve automatikusan számolja a lefolytatott órákat.
- **Diagnosztika:** A diagnosztika fülön valós időben követhető a „Rumble” (motorrezgés) és a „Sibilance” (magas frekvenciás torzítás) szintje, valamint a felületi kattogások száma.
- **Testreszabás:** A vizuális beállításoknál módosítható a felület témája, hogy illeszkedjen a fizikai lemez színéhez.

### 2.3.3 Külső Integrációk

A Last.fm naplózás beállításához a profilbe kell lépni, ahol a rendszer egy hitelesítési folyamat után összekapcsolódik a felhasználó fiókjával. Ezután minden lejátszott szám automatikusan rögzül a profilban.

## 3. fejezet

# Fejlesztői dokumentáció

### 3.1 A probléma részletes specifikációja

A rendszer alapvető célja egy olyan szoftveres megoldás létrehozása, amely képes egy analóg audio jelből zenei metaadatokat és diagnosztikai információkat kinyerni.

#### 3.1.1 Funkcionális követelmények

A rendszernek a következő funkciókat kell biztosítania:

- **Valós idejű audio azonosítás:** Folyamatos monitorozás, lejátszáskezdés detektálása és audio fingerprinting alapú azonosítás.
- **Metaadat-bővítés:** Albumborítók, pontos sávhosszúság és hivatalos albumnevek lekérése külső API-k segítségével.
- **Szinkronizált dalszöveg:** Időbélyeggel ellátott szövegek lekérése és a lejátszási időhöz szinkronizált megjelenítése.
- **Hardverdiagnosztika:** RMS hangerőszint mérése, felületi kattogások (clicks) detektálása, motorrezgés (rumble) és sibilance mérése.
- **Hangszedő életciklus kezelése:** Tűk kopásának nyomon követése és riasztás a maximális élettartam elérésekor.
- **Digitális naplózás:** Automatikusan szinkronizált zenehallgatási előzmények mentése a Last.fm profilba.

### 3.1.2 Nem-funkcionális követelmények

- **Teljesítmény:** A diagnosztikai adatoknak másodpercenként többször frissülnie kell a frontend felületen (alacsony latency).
- **Megbízhatóság:** Fallback mechanizmusok alkalmazása a külső API-k meghibásodása esetén.
- **Környezet:** A rendszernek Docker konténerben kell futnia a függőségek izolálása érdekében.

## 3.2 Felhasznált módszerek és fogalmak

### 3.2.1 Alapvető fogalmak

- **Audio Fingerprinting:** Az audiojel akusztikai jellemzőinek (spektrum csúcsok) digitális aláírásra alakítása és egy adatbázissal való összehasonlítása.
- **RMS (Root Mean Square):** Az audiojel effektív értéke, amely az átlagos energiát reprezentálja.
- **FFT (Fast Fourier Transform):** Gyors Fourier-transzformáció, amely lehetővé teszi a jel alakítását az időterületből a frekvenciaterületbe.

### 3.2.2 Alkalmazott módszerek

A rendszer egy kliens-szerver architektúrát használ, ahol a backend Python, a frontend pedig Vue.js. A valós idejű adatcsere WebSockets (Socket.IO) technológiával valósul meg, így elkerülhető a hagyományos HTTP polling késleltetése. A jelfeldolgozáshoz a NumPy és SciPy könyvtárakat alkalmazom, amelyek lehetővé teszik a sávszűrést (Butterworth filter) és a matematikai derivátumok számítását a kattogások detektálásához.

## 3.3 Szoftver architektúra és szerkezet

### 3.3.1 Logikai és fizikai szerkezet

A szoftver egy modularizált felépítéssel rendelkezik:

- **Audio Processor Modul:** A hardveres pufferból olvassa a jelet, számolja a diagnosztikát és detektálja a számkezdést.



- **Recognition Engine Modul:** Kezeli a fingerprinting API-k hívásait és a metaadat-vízesés (waterfall) stratégiát.
- **API Layer (Flask):** Biztosítja a REST végpontokat a konfigurációk kezeléséhez.
- **WebSocket Server:** Sugározza az élő telemetriát a kliensek felé.
- **Frontend SPA (Vue.js):** Vizualizálja az adatokat és kezeli a felhasználói interakciókat.

### 3.3.2 Modellezés és Adatbázis

A rendszer működését UML diagrammok és egy relációs adatmodell támogatja.

- **Use Case Modell:** A felhasználó és a rendszer között alakuló interakciókat szemlélteti (lásd 3.1).
- **Adatbázis Modell (ERD):** Az SQLite adatbázisban a `User`, `Cartridge`, `TrackHistory`, `AlbumColor` és `TrackOffset` táblák között normalizált kapcsolatok állnak (lásd 3.2).
- **Szekvenciagramm:** A többszálú feldolgozást és az aszinkron API hívások sorrendjét mutatja be (lásd 3.3).

## 3.4 Felhasználói felület tervezése

A felület egy reaktív Single Page Application (SPA). A tervezésnél a következő szempontokat tartottam szem előtt:

- **Reszponzivitás:** CSS konténer-lekérdezések és folyékony tipográfia használata a mobil- és asztali eszközök kompatibilitásához.
- **Dinamikus témázás:** A CSS változók segítségével a felület színei változnak az azonosított album borítójához vagy a fizikai lemez színéhez.
- **Interaktivitás:** WebSockets alapú VU Meter és egy görgethető, időbélyegezett dalszöveg-megjelenítő.

## 3.5 Tesztelési terv és eredmények

### 3.5.1 Tesztelési módszerek

A tesztelés során különböző típusú bakelitlemezeket (különböző műfajok és állapotok) használtam a rendszer validálására:

- **Integrációs teszt:** A hardveres bemenet és a szoftveres felismerő együttes működése.
- **Edge-case tesztelés:** Karcolt lemezek használata a click-detection algoritmus érzékenységének ellenőrzésére.
- **Teljesítményteszt:** A CPU és I/O terhelés mérése Raspberry Pi környezetben.

### 3.5.2 Tesztelési eredmények

A tesztelés során az alábbi eredményeket értem:

- **Felismerési pontosság:** A rendszer kb. 80% esetben azonosította a számokat első próbálkozásra.
- **Latency:** A diagnosztikai adatok megjelenítése 100ms késleltetéssel történik.
- **Optimalizálás:** A Write-Ahead Logging (WAL) bekapcsolása után az adatbázis-zárolási hibák teljesen megszűntek, az I/O terhelés jelentősen csökkent.

## 3.6 Mesterséges intelligencia alkalmazása a fejlesztés során

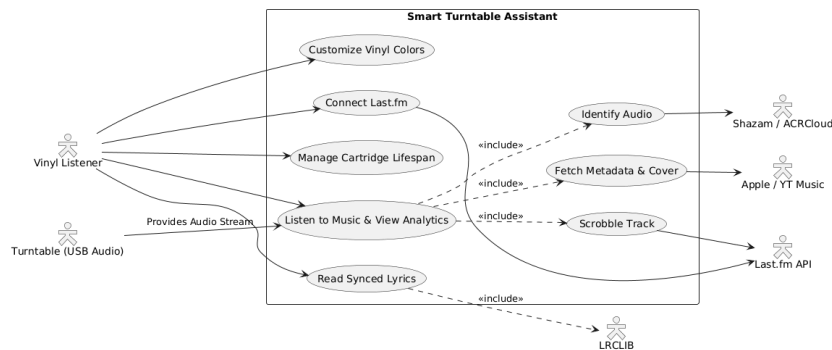
A projekt megvalósítása során modern nagynyelvi modellek (LLM) alkalmazása történt, amelyek elsődlegesen segítő eszközként, a fejlesztési folyamat gyorsítása és optimalizálása érdekében szolgáltak. Az AI használata a következő területeken történt:

- **Koncepciók elsajátítása és kutatás:** Ismeretlen technológiai területek (például a digitális jelfeldolgozás specifikus matematikai műveletei vagy komplex API integrációk) esetén az AI-t mint gyors referenciát használtam. A kapott példakódokat nem implementáltam

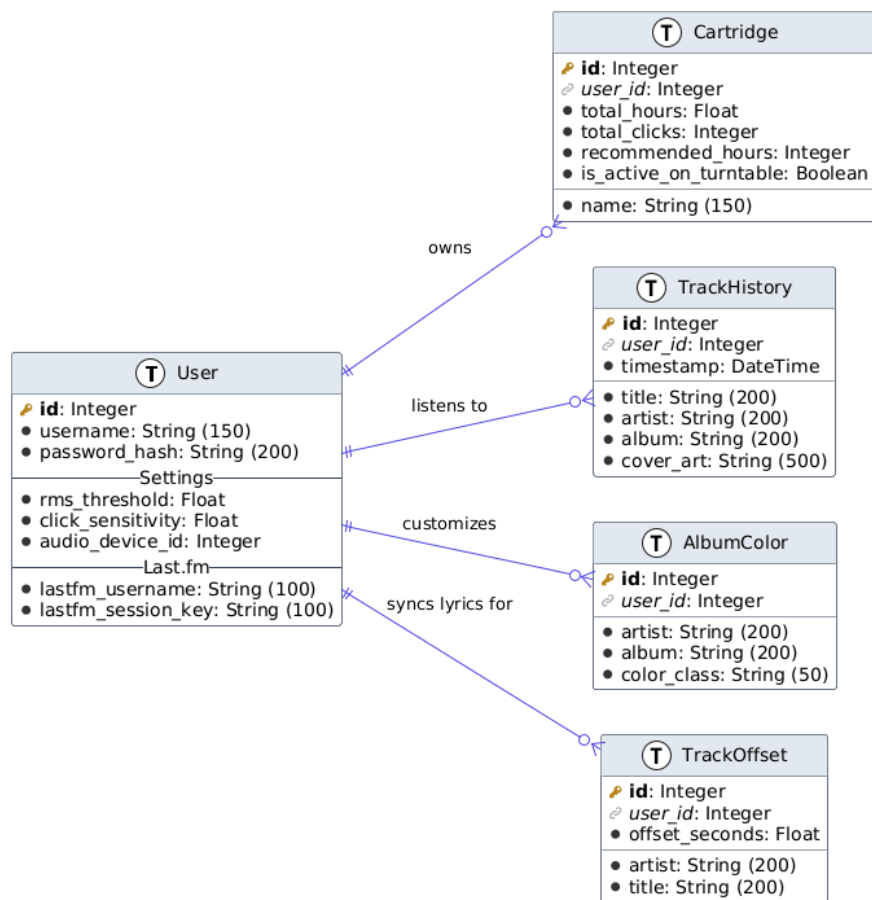
közvetlenül, hanem alapvető iránymutatónak tekintettem őket, amelyeket egy egyedi tervezési folyamat után adaptáltam a projekt specifikus igényeihez.

- **Ismétlődő kódstruktúrák (boilerplate) generálása:** A fejlesztés során felmerülő repetitív feladatok, mint például a frontend és backend közötti alapvető adatcsere logikája vagy a standard API végpontok vázlata, generálása során alkalmaztam AI segítséget. Ezek a kódrészletek egy alapvázaszt szolgáltattak, amelyekvégső formáját manuális átírással és finomhangolással érhettem el.
- **Hibakeresés és optimalizálás:** A debugolási folyamat során az AI-t kódsnippek elemzésére és potenciális hibaforrások gyorsabb azonosítására használtam. Mivel az AI nem ismerte a teljes projekt architektúráját, a javasolt megoldásokat kritikus szemlélettel szűrtem, és csak a projekt kontextusába illeszthető megoldásokat integráltam a végleges kódbázisba.

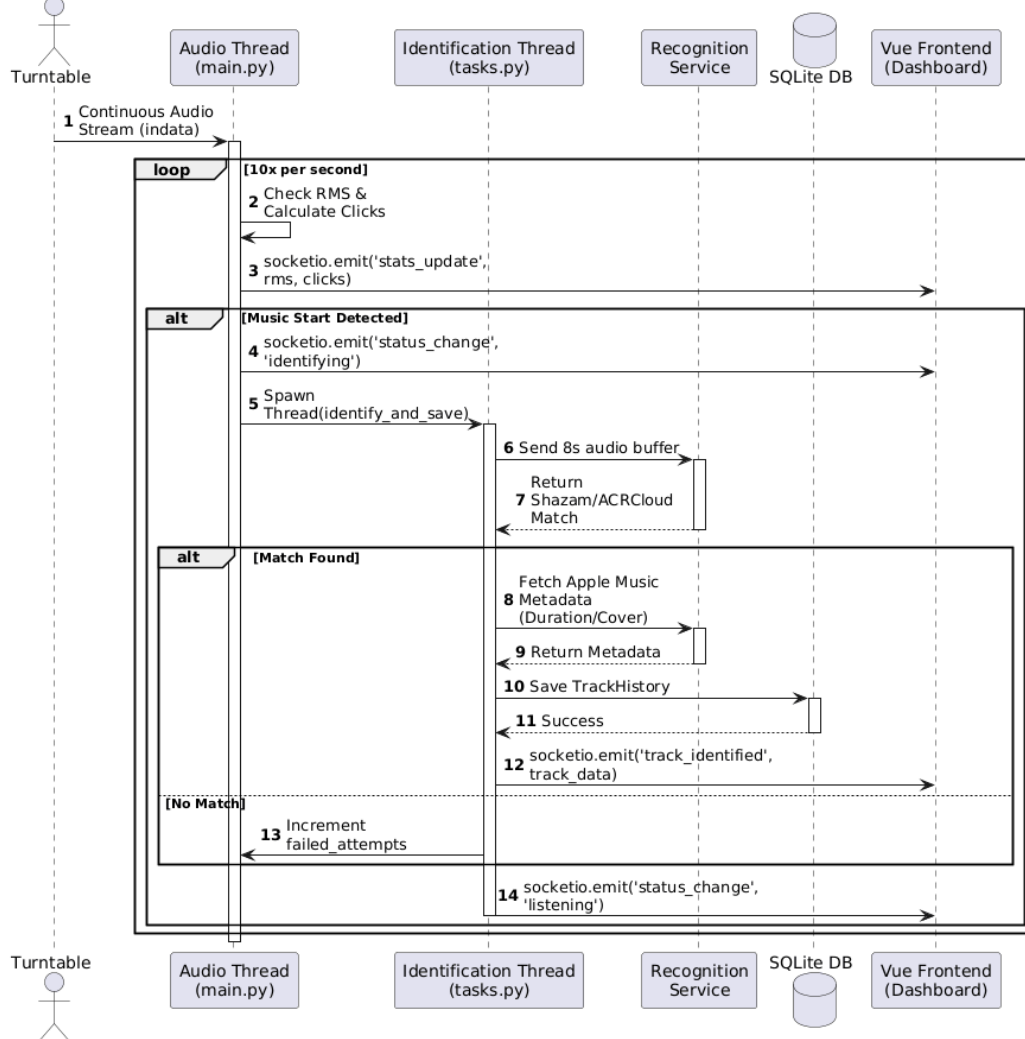
Külön kiemelném, hogy a rendszer architektúrájának tervezése, valamint a végleges szoftveres implementáció teljes egészében a szerző munkájának eredménye. Az AI használata kizárólag a fejlesztési ciklus hatékonyságának növelésére és a tanulási folyamat gyorsítására szolgált.



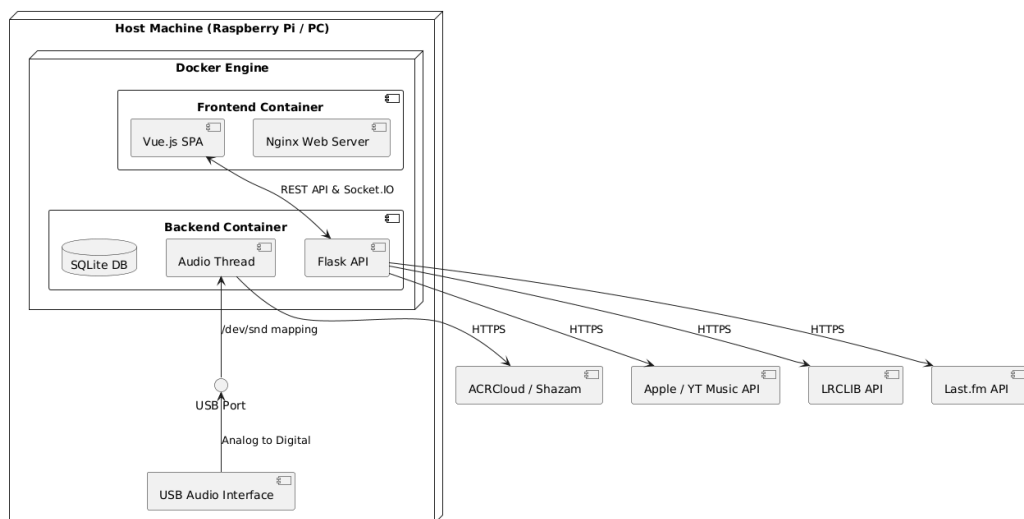
3.1. ábra: UML Use Case Diagram



3.2. ábra: ER Diagram az adatbázis sémájáról



3.3. ábra: UML Sequence Diagram



3.4. ábra: Deployment diagram amely a docker image felépítését mutatja be.